Project: Simulating the Ideal Gas through 3D Molecular Dynamics

Instructors: Shilpa Sattiraju & Mayank Agrawal

1 Objective

The goal of this project is to build a 3D Molecular Dynamics (MD) simulation of gas particles from first principles using Python. You will define the system using constant volume, constant number of particles and variable temperature. By the end of the project, you will measure macroscopic properties—**Pressure** and computationally derive the Ideal Gas Equation:

$$PV = nRT$$

2 The System Definition

You will simulate a 3D "Perfectly Elastic" environment. To model an **Ideal Gas**, the system must follow these constraints:

- **The Container:** A cubic volume with side length L (Total Volume $V = L^3$).
- **The Particles:** N identical point-particles with mass m and position vector $\mathbf{r} = (x, y, z)$. Please choose any value of L and N as per your preference.
- **Motion:** Particles move in straight lines at constant velocity $\mathbf{v} = (v_x, v_y, v_z)$ between wall collisions. The velocity of each particle must be defined using temperature.
- **Boundaries:** Perfectly elastic collisions with 6 walls. For example, when a particle hits a wall perpendicular to the x-axis, $v_x \to -v_x$.
- **Simplification:** Ignore inter-particle collisions and gravity.

3 TO DO Part I: Initializing Velocities for a Target Temperature

To initialize the system at a specific temperature T, velocities for each particle i must be drawn from a Maxwell-Boltzmann distribution.

1. **Random Sampling:** Assign each velocity component (v_x, v_y, v_z) using a normal distribution with a mean of 0 and a standard deviation of $\sigma = \sqrt{k_B T/m}$.
2. **Zeroing Momentum:** To prevent the gas from drifting, calculate the average velocity vector of the system and subtract it from each particle:

$$\mathbf{v}_{i,\text{corrected}} = \mathbf{v}_i - \frac{1}{N}\sum_{j=1}^{N} \mathbf{v}_j$$

3. **Temperature Rescaling:** To ensure the kinetic energy matches the target T exactly, scale all velocities by $\lambda = \sqrt{T_{\text{target}}/T_{\text{current}}}$, where T_{current} is the temperature calculated from the initial random speeds.

3.1 Calculating Temperature (T)

According to the Kinetic Theory of Gases, temperature is a measure of the average translational kinetic energy per molecule. In 3D:

$$\frac{3}{2}k_B T = \frac{1}{2}m\langle v^2 \rangle$$

You can calculate the mean squared speed $\langle v^2 \rangle = \frac{1}{N}\sum v_i^2$ of all particles to determine the system's temperature.

4 TO DO Part II: Microscopic Physics (The Engine)

Using the `NumPy` library, you must implement the update loop for every time step dt.

4.1 Kinematics

Update the position of all N particles simultaneously using vector addition:

$$\mathbf{r}_{\text{new}} = \mathbf{r}_{\text{old}} + \mathbf{v} \cdot dt$$

4.2 Wall Collisions & Momentum

When a particle's position exceeds the boundary $[0, L]$, it must bounce. During this bounce, the particle imparts an **Impulse** to the wall. The change in momentum is:

$$\Delta p = |2 \cdot m \cdot v_\perp|$$

where $v_\perp$ is the velocity component perpendicular to the wall hit.

5 TO DO Part III: Calculating Macroscopic Properties

You must write functions to calculate the state of the gas from the motion of individual particles.

5.1 Calculating Pressure (P)

Pressure is defined as the force exerted by particles divided by the surface area of the walls.

$$P = \frac{\sum \Delta p}{A \cdot \Delta t} = \frac{\sum \Delta p}{(6L^2) \cdot \Delta t}$$

Task: Over a fixed time interval Δt, sum all momentum changes ($\sum \Delta p$) from every wall collision and divide by the total surface area of the cube.

6 TO DO Part IV: Deriving the Ideal Gas Law

Run the simulation multiple times, varying one parameter (temperature) while keeping others constant. Calculate pressure each time.

6.1 The Final Analysis

By plotting your results, you will observe the following proportionalities:

$$P \propto T$$

Combine these to derive the final equation:

$$P = \text{constant} \cdot \frac{N \cdot T}{V}$$

Calculate the slope of your line to find your "Simulation Gas Constant."

7 Technical Requirements

- **Language:** Python 3.x
- **Libraries:** `NumPy` (for vectorized math), `Matplotlib` (for plotting).
- **Deliverable:** A script that generates a 3D animation of the gas and a graph proving the linear relationship between P and $\frac{NT}{V}$.